# INTERVIEW DEFENSE DOCUMENT

## DevPulse Platform v3.5

Production-simulated RAG + agentic migration intelligence platform.
Version-safe retrieval, deterministic conflict detection, LLM-last synthesis,
agentic goal planning, repo-aware callsite analysis, and patch simulation.

---

| Metric | Value |
|---|---|
| Wrong-version answer rate | 0.0 |
| Hybrid Recall@5 | 0.94 |
| Reranker Recall@5 | 0.97 |
| Conflict Macro F1 | 0.966 |
| 37-day backtest queries | 2,479 |
| RAG eval query count | 180 |
| Risky callsites found | 10/10 |
| Patch recommendation | DO_NOT_APPLY_WITHOUT_REVIEW |
| Evidence artifacts | 70 |
| Failure/recovery scenarios | 19 |

Sidharth Kriplani | May 2026

## Section 1 — Hard Interview Q&A

**Q1 — Wrong-version rate is 0.0. How is that achieved, and under what conditions would it fail?**
Version-safe retrieval uses hard version filtering: the retrieval step only returns chunks whose metadata version field matches (or is compatible with) the version specified in the query. A query for 'requests v3 timeout handling' will not retrieve a chunk tagged as v2. The wrong-version rate is 0.0 on the 180-query eval set because every query in that set specifies a version, and the index has clean version metadata. The conditions under which it would fail: (1) a document chunk has an incorrect or missing version tag — the filter would either return nothing (false negative) or return a wrong-version chunk if the filter logic is permissive; (2) a query implicitly assumes a version without stating it (e.g., 'how do I use timeout in requests') — the query parser would need to infer the version from context, which is not deterministically correct; (3) version ranges (e.g., >=v2) require range-aware filtering that is more complex than exact match.

**Q2 — The conflict detector achieves Macro F1 = 0.966 on four conflict types. What are the four types and what is the hardest to detect?**
The four conflict types are: Stale (a chunk references a deprecated API or behaviour that has been superseded in a later version), Contradictory (two chunks from different sources make incompatible claims about the same API surface), Deprecated (a chunk explicitly marks an API as deprecated but a newer chunk using it exists in the index), and Cross-source (two sources — e.g., official docs and a StackOverflow snippet — disagree on behaviour). The hardest to detect is Contradictory because it requires comparing two retrieved chunks for semantic incompatibility, not just checking a single chunk against metadata. The current implementation uses deterministic pattern matching: if two chunks cover the same API endpoint/method signature but return different parameter descriptions, it is flagged as contradictory. Purely paraphrase-based contradictions (two different wordings of incompatible behaviour) are not reliably detected by pattern matching and require LLM-based conflict resolution.

**Q3 — The LLM-Last design means synthesis only happens when evidence is grounded. What does 'grounded' mean operationally?**
Grounded means the conflict detection stage returned zero CONTRADICTORY or STALE conflicts, and the retrieval stage returned at least one version-matched chunk with a confidence score above the synthesis threshold. Operationally: (1) no conflicts detected among retrieved chunks, (2) at least K=3 chunks retrieved that match the query version, (3) none of the chunks is flagged as deprecated for the queried version, (4) the query complexity routing did not classify the query as requiring multi-hop reasoning beyond the retrieved evidence. When any condition fails, the system returns BLOCKED (no synthesis, evidence insufficient) or RISKY (synthesis with explicit caveats about conflict or low coverage). This prevents the LLM from generating a confident answer when the retrieval layer returned stale or contradictory evidence — the class of failure that causes production migrations to break.

**Q4 — Goal Mode has bounded retry. What is the retry logic and why is it bounded?**
Task execution retries a failed step up to a configurable cap (default: 3 retries per step). After exceeding the cap, RecoveryDecider escalates — it does not retry again. Escalation means: (1) the step is marked as unresolvable in the plan, (2) the PlanSummaryReporter marks the overall migration as BLOCKED at that step, (3) a structured escalation artifact is written with the step ID, the failure type, and the suggested human

intervention. The bound is essential because an unbounded retry loop is the most common source of runaway agentic pipelines. A genuine step failure (e.g., the API signature has changed and no valid migration path exists) will never succeed no matter how many retries are attempted. Bounded retry distinguishes transient failures (network timeout, rate limit) from structural failures (API removed) and surfaces the latter for human resolution rather than spinning indefinitely.

**Q5 — The repo-aware analysis found 10 risky callsites and recommended DO_NOT_APPLY_WITHOUT_REVIEW for all 10. When would a callsite be SAFE_TO_APPLY?**

SAFE_TO_APPLY requires three conditions to hold simultaneously: (1) the callsite uses the API in a way that is explicitly documented as backward-compatible in the target version's migration notes, (2) the conflict detector found no version-specific conflicts in chunks describing that API surface, and (3) the patched code compiles and passes simulated tests in the test simulation step. In the current corpus (requests v2 → v3 migration), all 10 callsites fail condition (1) because requests v3 made breaking changes to timeout signature, session handling, and exception hierarchy — none of which are backward-compatible drops-in. A SAFE_TO_APPLY callsite would be, for example, a callsite using requests.get() with only positional arguments where the signature is unchanged between v2 and v3 and no exception handling around v2-specific exceptions exists.

**Q6 — The 37-day backtest runs 2,479 queries. How is the backtest structured and what does it validate?**

The 37-day backtest replays simulated daily query loads against the versioned documentation index. Each day's batch of queries covers a mix of complexity levels (simple lookup, migration planning, conflict resolution) drawn from a deterministic seed. The backtest validates: (1) wrong-version rate stability across days (expected: 0.0 throughout), (2) recall@5 stability — the retrieval layer should not degrade as the index accumulates more documents, (3) conflict detection consistency — the same query on the same index should produce the same conflict classification on every run, (4) goal mode completion rate — what fraction of migration goals reach COMPLETED vs BLOCKED vs ESCALATED. The 2,479 queries provide coverage across the 37 days at ~67 queries/day. The backtest is deterministic (fixed RNG seed) so re-runs produce identical results, enabling comparison across code versions.

**Q7 — Patch simulation recommends DO_NOT_APPLY_WITHOUT_REVIEW. Why not REJECT or APPROVE?**

APPROVE would imply automated application is safe — which the system cannot certify without real test execution against the production codebase. REJECT would imply the migration is impossible — which is also not true. DO_NOT_APPLY_WITHOUT_REVIEW is the correct label for the situation: the patch diff is generated, the test simulation shows the before/after behaviour, the triage identifies which test cases would be affected, and the rollback plan is documented — but a human reviewer must verify that the simulated test outcomes match the actual test suite and that no edge cases in the production codebase are missed. This is the same decision boundary used by automated code review tools (e.g., Dependabot): they propose changes and flag risk, but require human approval before merge.

**Q8 — The system architecture has a Query Mode and a Goal Mode. When does the system route to Goal Mode?**

The query parser classifies input into simple retrieval (Query Mode) vs migration goal (Goal Mode) based on query structure. Indicators for Goal Mode routing: (1) the input contains an explicit goal verb ('migrate', 'upgrade', 'move from X to Y', 'replace X with Y'), (2) the input references two version states (current and target), (3) the dependency delta detector identifies a version difference between the stated current and target that implies code changes are required. Simple retrieval queries ('What is the timeout parameter in requests v3?') stay in Query Mode — they need a single-turn lookup with conflict-checked synthesis. Goal Mode is heavier: it generates a multi-step plan, executes each step with retry/recovery, and produces a structured migration report. Routing incorrectly to Goal Mode for a simple query would generate unnecessary overhead; routing to Query Mode for a migration goal would fail to produce the plan.

**Q9 — The citation assembly step in LLM-Last synthesis produces chunk-level citations. What are these used for?**

Citations serve two purposes. First, answer auditability: every claim in the synthesised migration guidance is traceable to a specific chunk ID, source document, and version. A reviewer can verify that 'use timeout=(connect, read) tuple in requests v3' is sourced from the official requests v3 changelog, not hallucinated. Second, staleness detection in production: if the source document for a cited chunk is updated (new version released), the citation can be flagged for re-evaluation — the answer may no longer be correct. The fallback audit artifact records cases where synthesis was blocked because citations were insufficient or conflicting. In a production deployment, citation-backed answers would be the only answers surfaced to developers; any unciteable claim would trigger BLOCKED status.

**Q10 — What does DevPulse NOT do that a production migration intelligence platform would require?**

Six things. First, real code execution — test simulation is deterministic pattern matching, not actual pytest/unittest execution against the codebase. Second, live index updates — the documentation index is static; a production system would ingest changelogs, release notes, and GitHub issues in real time. Third, multi-repo awareness — the repo scanner analyses a single local repository; production systems need to handle monorepos and service meshes with dozens of interdependent packages. Fourth, CI/CD integration — there is no GitHub Actions or Jenkins hook; a production system would trigger migration analysis on dependency file changes (requirements.txt, pyproject.toml). Fifth, team-level workflows — no PR review, approvals, or comment threading. Sixth, transitive dependency analysis — the system analyses direct dependency API usage; breaking changes in transitive dependencies (library A uses library B internally) are not surfaced.

## Section 2 — What DevPulse Does Not Claim

**No real code execution.** Test simulation uses deterministic pattern matching. Actual pytest/unittest execution against a live codebase is not performed.

**No live index updates.** Documentation index is static. Production would ingest changelogs and release notes in real time.

**No CI/CD integration.** No GitHub Actions hook or Jenkins pipeline integration. Migration analysis is triggered manually.

**No real LLM in production path.** LLM-Last synthesis is simulated. The boundary logic (grounding checks, conflict gates) is implemented; LLM API calls are stubs.

**No multi-repo transitive analysis.** Direct dependency API changes are analysed. Transitive dependency breaking changes are not surfaced.

**No team workflow.** No PR review, approval workflow, or comment threading. Single-user analysis only.